

INGENIERÍA DE DATOS II

Tecnicatura / Licenciatura / Ingeniería en Informática

TRABAJO PRÁCTICO INTEGRADOR 1.ª ENTREGA

TEMA 8

Plataforma de Streaming de Música

Motores requeridos	MongoDB · Neo4j
Capa de integración	Persistencia Poliglota (MongoDB + Neo4j) — 2.a Entrega
Unidades evaluadas	I — Relacional vs NoSQL · II — Modelos NoSQL · III — Modelado NoSQL
Fecha de entrega	Lunes 20 de abril de 2025
Modalidad	Grupal (5 integrantes)
Defensa oral	Luego de corregidas ambas entregas (fecha a confirmar)

1. Contexto y Descripción del Problema

Una plataforma de streaming de música cuenta con 20 millones de usuarios activos mensuales, un catálogo de 80 millones de canciones y registra 400 millones de reproducciones diarias. El sistema actual presenta los siguientes problemas críticos:

- El registro de reproducciones genera 400 millones de eventos diarios. El sistema relacional no puede absorber ese volumen de escritura sin pérdida de datos.
- El motor de recomendaciones necesita recorrer redes de relación artista–colaboración–influencia que en SQL requieren 8 o más joins y tardan minutos en ejecutarse.
- Las playlists tienen estructura variable y pueden ser generadas por usuarios, por algoritmos o por editores, con metadatos distintos en cada caso.
- La búsqueda por similitud musical (artistas parecidos, géneros relacionados, estados de ánimo) no puede modelarse eficientemente en tablas planas.

El grupo deberá diseñar e implementar un sistema NoSQL que resuelva estos problemas utilizando MongoDB y Neo4j. Esta entrega cubre el modelado y las consultas de cada motor de forma independiente. Cada decisión de diseño debe estar justificada técnicamente en el informe.

2. Dominio del Sistema

2.1 Entidades principales

El sistema debe gestionar las siguientes entidades de negocio:

- **Usuario:** nombre, país, plan (gratis / premium), géneros favoritos, artistas seguidos, historial de reproducciones.
- **Artista:** nombre artístico, género principal, país de origen, seguidores, discografía.
- **Álbum:** nombre, artista, año de lanzamiento, género, sello discográfico, canciones.
- **Canción:** nombre, artista(s), álbum, duración, géneros, BPM, estado de ánimo (mood), popularidad.
- **Reproducción:** usuario, canción, timestamp, dispositivo, contexto (playlist / búsqueda / radio / álbum), completada (sí/no).
- **Playlist:** nombre, tipo (usuario / editorial / algorítmica), canciones ordenadas, seguidores, descripción.
- **Colaboración:** artistas involucrados, canción o álbum resultante, tipo (featuring / producción / composición).

2.2 Relaciones clave

Las siguientes relaciones son centrales en el dominio y deben ser modeladas con cuidado:

- Un usuario puede seguir a múltiples artistas y guardar canciones en su biblioteca.

- Un artista puede colaborar con otros en canciones o producciones, formando una red de influencias.
- Dos artistas están relacionados si comparten colaboradores en común, aunque nunca hayan trabajado directamente.
- Las canciones de géneros similares forman clusters en el grafo que permiten la recomendación por proximidad.
- Un usuario se parece a otro si tienen alta superposición en su historial de reproducciones.

3. Requerimientos del Sistema

3.1 Requerimientos de datos (MongoDB)

El grupo deberá implementar en MongoDB el catálogo musical, las playlists y el historial de reproducciones. A continuación se listan los requerimientos mínimos:

1. Diseñar el esquema de colecciones para: usuarios, artistas, canciones, reproducciones y playlists.
2. Justificar explícitamente en el informe las decisiones de embedding vs. referencia para cada relación relevante.
3. Implementar al menos 3 índices que mejoren el rendimiento de las consultas más frecuentes e indicar en el informe qué consulta optimiza cada índice.
4. Desarrollar y documentar las siguientes consultas funcionales:
 - a) Obtener las 10 canciones más reproducidas globalmente en las últimas 24 horas, con nombre del artista y álbum.
 - b) Dado un usuario, obtener su historial de reproducciones de la última semana agrupado por artista y género.
 - c) Encontrar canciones con alta cantidad de reproducciones pero baja tasa de completitud (los usuarios las cortan antes de terminar).
 - d) Listar las playlists con mayor crecimiento de seguidores en el último mes.
 - e) Para un usuario premium, obtener recomendaciones de álbumes completos basadas en artistas que escucha frecuentemente.
5. Cargar un conjunto de datos de prueba coherente: mínimo 500 canciones, 80 artistas, 20 álbumes, 200 usuarios, 3.000 reproducciones distribuidas en 7 días.

3.2 Requerimientos de grafos (Neo4j)

Neo4j debe utilizarse para modelar la red de colaboraciones entre artistas, las relaciones de influencia musical y el motor de recomendación. Requerimientos mínimos:

6. Diseñar el grafo de nodos y relaciones. Definir explícitamente: nodos, etiquetas, propiedades y tipos de relación con su dirección y semántica.
7. Implementar y documentar las siguientes consultas en Cypher:

- a) Dado un artista, encontrar todos los artistas conectados en hasta 2 saltos de colaboración directa.
 - b) Encontrar el camino de colaboraciones entre dos artistas que nunca trabajaron juntos directamente.
 - c) Detectar artistas que son 'puentes' entre géneros distintos: colaboraron con artistas de al menos 3 géneros diferentes.
 - d) Dado un usuario, recomendar artistas escuchados por usuarios con historial similar que el usuario aún no conoce.
 - e) Encontrar clusters de artistas altamente interconectados (comunidades musicales) e identificar el género dominante de cada cluster.
- 8. Justificar por qué estas consultas son más adecuadas en un modelo de grafos que en un modelo documental o relacional.
 - 9. Cargar datos de prueba coherentes con los de MongoDB (al menos las mismas entidades principales compartidas entre ambos motores).

4. Informe Escrito — Estructura Obligatoria

El informe es parte central de la nota. Debe tener la siguiente estructura mínima:

- 10. **Introducción:** descripción del problema, tecnologías elegidas y justificación inicial de por qué MongoDB y Neo4j para este dominio.
- 11. **Modelo de datos en MongoDB:** diagrama de colecciones, decisiones de embedding vs. referencia justificadas, índices implementados y consulta que optimiza cada uno.
- 12. **Modelo de datos en Neo4j:** diagrama del grafo (nodos, etiquetas, relaciones, propiedades), justificación del modelo y comparación con el modelo documental.
- 13. **Comparación relacional vs. NoSQL:** para al menos 2 consultas, mostrar cómo se resolvería en SQL y cómo en el motor elegido. Analizar ventajas y limitaciones de cada enfoque.
- 14. **Dificultades y decisiones de diseño:** qué problemas encontraron, qué alternativas evaluaron y por qué eligieron la solución final.
- 15. **Conclusiones:** reflexión crítica sobre el uso de cada tecnología y qué ventajas aporta cada motor al dominio del problema.
- 16. **Bibliografía:** obligatorio citar al menos Harrison (2015), Pivert (2018) y la documentación oficial de MongoDB y Neo4j.

Nota sobre la capa poliglota

La integración de ambos motores en una capa de persistencia poliglota será el eje de la Segunda Entrega. En esta primera entrega, MongoDB y Neo4j se implementan y documentan de forma independiente. Sin embargo, los datos de prueba deben ser coherentes entre ambos motores, ya que serán la base de la integración futura.

5. Defensa Oral

¿Cuándo es la defensa?

La defensa oral se realizará una vez aprobadas y corregidas ambas entregas del TP (1.a y 2.a). En ella se evalúa el trabajo completo e integrado, incluyendo la capa poliglota de la Segunda Entrega. La fecha exacta será comunicada por el docente luego de la corrección de la Segunda Entrega.

La defensa es obligatoria para aprobar el TP. Incluirá:

- **Demo en vivo del sistema completo:** la capa poliglota debe estar funcionando y conectada a todos los motores del TP.
- **Consultas en vivo:** el docente solicitará ejecutar consultas específicas en cualquiera de los motores implementados durante el TP.
- **Modificación de código en el momento:** puede solicitarse modificar o escribir una consulta nueva durante la defensa. Todos los integrantes deben ser capaces de hacerlo.
- **Preguntas conceptuales:** sobre decisiones de diseño, trade-offs evaluados y alternativas descartadas en ambas entregas.

Importante — Evaluación individual dentro del grupo

La defensa es individual: todos los integrantes deben conocer el sistema completo en profundidad. No alcanza con que 'alguien del grupo lo hizo'. Si un integrante no puede responder ni operar el sistema, su nota individual puede diferir de la nota grupal de las entregas.

6. Bibliografía de Referencia

Obligatoria

- Harrison, G. (2015). Next Generation Databases: NoSQL, NewSQL, and Big Data. Apress.
- Pivert, O. (Ed.). (2018). NoSQL Data Models: Trends and Challenges. ISTE.
- Deka, G. C. (2017). NoSQL: Database for Storage and Retrieval of Data in Cloud. CRC Press.

Documentación oficial

- MongoDB, Inc. (s/f). MongoDB Documentation. <https://www.mongodb.com/docs/>
- Neo4j, Inc. (s/f). Neo4j Documentation. <https://neo4j.com/docs/>